

WEEK – 6

CELEBAL EXCELLENCE INTERNSHIP

Topic: Spark Architecture and Efficient Data Processing

Using the `fake_retail_data.csv` used in the previous week.

Q1: Explain the roles of the **Driver**, **Cluster Manager**, and **Executor** in a Spark application.

Answer: Apache Spark follows a distributed architecture consisting of three main components:

Driver

- Acts as the main control program.
- Creates `SparkSession`.
- Converts user code into tasks.
- Sends tasks to executors.

Cluster Manager

- Manages resources across the cluster.
- Allocates CPU and memory resources.

Executors

- Run tasks assigned by the Driver.
- Store data in memory.
- Return results to the Driver.

Insight

The Driver coordinates execution, the Cluster Manager manages resources, and Executors perform computations.

Q2: How does Spark's **Lazy Evaluation** strategy improve performance when chain-processing large datasets?

Answer: Spark follows Lazy Evaluation, meaning transformations such as `filter()` and `select()` are not executed immediately.

Spark records all transformations in a DAG (Directed Acyclic Graph) and executes them only when an action such as `show()`, `count()`, or `collect()` is called.

Benefits

- Reduces unnecessary computations.
- Optimizes execution plans.
- Improves overall performance.

Example

```
[21]
✓ Os
filtered_df = df.filter(df.price > 100)
selected_df = filtered_df.select("user_id", "price")

selected_df.show()
```

user_id	price
3765	975.1
7644	141.56
2798	794.53
2489	1889.22
4222	1545.51
9114	1214.54
6670	229.38
7494	481.22
8856	980.16
2892	1841.22
4393	1713.97
4088	1244.97
5132	1111.31
3787	610.79
7072	1876.37
3940	1588.14
2578	1777.55
1140	858.9
2792	1066.97
3297	1560.53

Execution starts only at show().

Q3: Write a Spark command to read a CSV file located at "data/source.csv", ensuring the first row is treated as a header and inferSchema is enabled.

Q3

```
[22]
✓ Os
df = spark.read.csv(
    "fake_retail_data.csv",
    header=True,
    inferSchema=True
)

df.show(5)
```

user_id	transaction_date	region	product_category	sale_amount	city	age	subscription	email	username	price	store_id
3765	2025-11-12	North	Clothing	660.18	Pune	26	Basic	pjacobs@example.org	hugheswilliam	975.1	S104
7644	2026-03-10	West	Furniture	1572.95	Pune	19	Basic	NULL	karencruz	141.56	S103
2798	2026-01-01	North	Electronics	4365.53	Bangalore	22	Basic	NULL	terri58	794.53	S103
2489	2025-10-23	North	Books	4344.35	Mumbai	21	Basic	jmann@example.net	vharvey	1889.22	S104
4222	2026-04-23	West	Books	2506.72	Pune	25	Premium	chall@example.net	NULL	1545.51	S104

only showing top 5 rows

Q4: What is the difference between **CSV** and **Parquet** in terms of storage (row-based vs. columnar) and why does it matter for performance?

Answer:

CSV	Parquet
Row-based	Column-based
Larger storage	Smaller storage
Slower queries	Faster queries
No schema metadata	Stores schema

Why Performance Improves

Parquet reads only required columns.

Example:

```
df.select("price")
```

Spark loads only the price column.

Q5: Given a DataFrame df, write a query to select the columns product_id and price where the category is 'Electronics'.

```
[24]
✓ Os df.filter(
      col("category") == "Electronics"
    ).select(
      "product_id",
      "price"
    ).show()
```

product_id	price
2798	794.53
5475	1393.44
6359	668.92
2163	1271.89
2202	NULL
3932	1974.74
3730	1163.69
4089	1362.64
9525	1597.04
1887	246.09
7758	NULL
4745	1849.03
9814	1416.97
3416	216.47
5800	1157.18
2837	85.01
5955	564.38
9258	299.24
6054	888.49
4745	1849.03

Q6: Write the code to "revise" a DataFrame by renaming the column `old_name` to `new_name` and casting the price column from a String to a Double.

```
[11] ✓ Os
from pyspark.sql.types import DoubleType

df_new = (
    df.withColumnRenamed(
        "category",
        "new_name"
    )
    .withColumn(
        "price",
        col("price").cast(DoubleType())
    )
)

df_new.printSchema()
```

root

- user_id: integer (nullable = true)
- transaction_date: date (nullable = true)
- region: string (nullable = true)
- new_name: string (nullable = true)
- sale_amount: double (nullable = true)
- city: string (nullable = true)
- age: integer (nullable = true)
- subscription: string (nullable = true)
- email: string (nullable = true)
- username: string (nullable = true)
- price: double (nullable = true)
- store_id: string (nullable = true)
- raw timestamp: timestamp (nullable = true)

Q7: How does Spark use the **Lineage Graph (DAG)** to provide fault tolerance if a worker node fails?

Ans. Spark maintains a Lineage Graph (DAG) that records all transformations applied to the dataset.

If a worker node fails, Spark uses the DAG to recompute only the lost partitions instead of reprocessing the entire dataset.

Insight

This makes Spark fault-tolerant and efficient for large-scale distributed processing.

Q8: Write a query to filter a DataFrame `df_orders` for rows where the status is 'Completed' AND the amount is greater than 1000.

Q8

[25]
✓ Os

```
from pyspark.sql.functions import when, col

df_orders = df.withColumn(
    "status",
    when(col("sale_amount") > 1000, "Completed")
    .otherwise("Pending")
)
```

[26]
✓ Os

```
df_orders.filter(
    (col("status") == "Completed") &
    (col("sale_amount") > 1000)
).show()
```

▼

user_id	transaction_date	region	category	sale_amount	city	age	subscription	email	username	price	store_id	raw_timestamp	s
7644	2026-03-10	West	Furniture	1572.95	Pune	19	Basic	NULL	karencruz	141.56	S103	1995-08-26 21:08:...	Comp
2798	2026-01-01	North	Electronics	4365.53	Bangalore	22	Basic	NULL	terri58	794.53	S103	2003-09-10 13:51:...	Comp
2489	2025-10-23	North	Books	4344.35	Mumbai	21	Basic	jmann@example.net	vharvey	1889.22	S104	2005-02-22 01:52:...	Comp
4222	2026-04-23	West	Books	2506.72	Pune	25	Premium	chall@example.net	NULL	1545.51	S104	2015-07-13 22:09:...	Comp
9114	2025-12-25	South	Books	4860.95	Pune	28	Basic	mark81@example.com	jeffrey21	1214.54	S104	1984-05-12 22:30:...	Comp
7494	2025-06-11	East	Books	2312.17	Delhi	19	Premium	benjamincraig@exa...	tammyewing	481.22	S101	1997-11-06 10:25:...	Comp
8856	2026-04-18	North	Furniture	1703.4	Delhi	29	Basic	brownbrett@exempl...	wfoster	980.16	S101	1980-09-26 15:35:...	Comp
2892	2026-04-19	North	Books	3665.74	Delhi	25	Basic	sgarcia@example.net	wthomas	1841.22	S101	2009-02-28 13:26:...	Comp
4393	2026-01-16	North	Furniture	2839.23	Bangalore	58	Basic	tcook@example.com	NULL	1713.97	S104	2007-07-22 16:06:...	Comp

Q9: Explain the concept of **Predicate Pushdown** in Parquet and how it affects the amount of data loaded into memory.

Answer: Predicate Pushdown is an optimization technique where filter conditions are pushed down to the storage layer.

Instead of reading the entire file:

```
df.filter(
    col("region") == "North"
)
```

Spark reads only matching records.

Benefits

- Less memory usage
- Faster execution
- Reduced disk I/O

Q10: Write a code snippet to add a new column `final_price` which is the `base_price` multiplied by 1.18 (18% tax).

```
[30]
✓ Os
from pyspark.sql.functions import col
from pyspark.sql.types import DoubleType

df = df.withColumn(
    "base_price",
    col("base_price").cast(DoubleType())
)

df = df.withColumn(
    "final_price",
    col("base_price") * 1.18
)

df.select(
    "base_price",
    "final_price"
).show(10)
```

base_price	final_price
975.1	1150.618
141.56	167.0408
794.53	937.5454
1889.22	2229.2796
1545.51	1823.7017999999998
1214.54	1433.1571999999999
229.38	270.66839999999996
481.22	567.8396

Q11: What is the difference between **Transformations** and **Actions**? Provide two examples of each.

Answer

Transformations

- Create a new DataFrame.
- Executed lazily.

Examples:

```
df.filter()
df.select()
```

Actions

- Trigger execution.

Examples:

```
df.show()
df.count()
```

Insight

Transformations build the execution plan, while actions execute it.

Q11

```
[32]
✓ 1s

# Transformation
filtered_df = df.filter(col("price") > 500)

# Action
filtered_df.show(5)

# Action
print(filtered_df.count())
```

user_id	transaction_date	region	category	sale_amount	city	age	subscription	email	username	price
3765	2025-11-12	North	Clothing	660.18	Pune	26	Basic	pjacobs@example.org	hugheswilliam	975.1
2798	2026-01-01	North	Electronics	4365.53	Bangalore	22	Basic	NULL	terri58	794.53
2489	2025-10-23	North	Books	4344.35	Mumbai	21	Basic	jmann@example.net	vharvey	1889.22
4222	2026-04-23	West	Books	2506.72	Pune	25	Premium	chall@example.net	NULL	1545.51
9114	2025-12-25	South	Books	4860.95	Pune	28	Basic	mark81@example.com	jeffrey21	1214.54

only showing top 5 rows
83

Q12: Write the Spark command to load a Parquet file from "path/to/input", filter out any rows where user_id is null, and save the result as a CSV at "path/to/output".

```
[34]
✓ 1s

from pyspark.sql.functions import col

df_parquet = spark.read.parquet(
    "input_parquet"
)

clean_df = df_parquet.filter(
    col("user_id").isNotNull()
)

clean_df.write.mode("overwrite").csv(
    "output_csv",
    header=True
)

clean_df.show(5)
```

user_id	transaction_date	region	category	sale_amount	city	age	subscription	email	username	price	store_id	raw_timestamp	status	product
3765	2025-11-12	North	Clothing	660.18	Pune	26	Basic	pjacobs@example.org	hugheswilliam	975.1	S104	1972-03-03 19:10:...	Pending	37
7644	2026-03-10	West	Furniture	1572.95	Pune	19	Basic	NULL	karencruz	141.56	S103	1995-08-26 21:08:...	Active	76
2798	2026-01-01	North	Electronics	4365.53	Bangalore	22	Basic	NULL	terri58	794.53	S103	2003-09-10 13:51:...	Inactive	27
2489	2025-10-23	North	Books	4344.35	Mumbai	21	Basic	jmann@example.net	vharvey	1889.22	S104	2005-02-22 01:52:...	Active	24
4222	2026-04-23	West	Books	2506.72	Pune	25	Premium	chall@example.net	NULL	1545.51	S104	2015-07-13 22:09:...	Active	42

only showing top 5 rows

Q13: In Spark Architecture, what is the difference between **Client Mode** and **Cluster Mode**?

Client Mode	Cluster Mode
Driver runs on local machine	Driver runs inside cluster
Used for development	Used for production
Less fault tolerant	More fault tolerant

Q14: Write a query to filter a dataset for rows where the region is 'North' OR the priority is 'High'.

Q14

```
[35]
✓ Os
from pyspark.sql.functions import col

df.filter(
    (col("region") == "North") |
    (col("priority") == "High")
).show()
```

	user_id	transaction_date	region	category	sale_amount	city	age	subscription	email	username	price	store_id	raw_timestamp	status
	3765	2025-11-12	North	Clothing	660.18	Pune	26	Basic	pjacobs@example.org	hugheswilliam	975.1	S104	1972-03-03 19:10:...	Pending
	2798	2026-01-01	North	Electronics	4365.53	Bangalore	22	Basic	NULL	terri58	794.53	S103	2003-09-10 13:51:...	Inactive
	2489	2025-10-23	North	Books	4344.35	Mumbai	21	Basic	jmann@example.net	vharvey	1889.22	S104	2005-02-22 01:52:...	Active
	4222	2026-04-23	West	Books	2506.72	Pune	25	Premium	chall@example.net	NULL	1545.51	S104	2015-07-13 22:09:...	Active
	9114	2025-12-25	South	Books	4860.95	Pune	28	Basic	mark81@example.com	jeffrey21	1214.54	S104	1984-05-12 22:30:...	Pending
	8856	2026-04-18	North	Furniture	1703.4	Delhi	29	Basic	brownbrett@exampl...	wfoster	980.16	S101	1980-09-26 15:35:...	Active
	2892	2026-04-19	North	Books	3665.74	Delhi	25	Basic	sgarcia@example.net	wthomas	1841.22	S101	2009-02-28 13:26:...	Inactive
	4393	2026-01-16	North	Furniture	2839.23	Bangalore	58	Basic	tcook@example.com	NULL	1713.97	S104	2007-07-22 16:06:...	Inactive
	1036	2025-07-06	South	Books	2728.63	Delhi	25	Basic	kbass@example.net	robertreeves	NULL	S103	2010-08-07 06:55:...	Inactive
	3787	2025-08-21	North	Books	3188.87	Delhi	54	Premium	icarpen@exampl...	michelle50	610.79	S104	2013-04-09 01:17:...	Pending
	7072	2026-05-10	West	Furniture	2651.24	Bangalore	32	Premium	alexander79@examp...	virginiaivincen	1876.37	S101	2000-10-05 11:31:...	Pending
	3940	2026-01-06	North	Books	788.36	Mumbai	22	Premium	sarah08@example.net	jonathanmullins	1588.14	S104	1983-09-02 16:54:...	Active
	1140	2025-11-24	North	Furniture	3224.59	Pune	41	Premium	garrett73@example...	hardinphilip	858.9	S103	1998-02-10 13:36:...	Inactive
	2792	2025-10-20	North	Clothing	3270.68	Delhi	38	Premium	ocox@example.net	ashley18	1066.97	S104	2022-01-29 00:02:...	Inactive

Q15: When exploring a dataset, why is it safer to use .show(5) instead of .collect() on a multi-terabyte dataset?

Answer

.show(5) displays only a few rows.

.collect() transfers the entire dataset to the Driver.

For large datasets, collect() may cause memory issues and application crashes.

Example

df.show(5)

Insight

Always use show() when exploring large datasets.